

CARMEN

Salud cardiaca

Manual de uso y estado del sistema

Monitoreo cardiovascular domiciliario
con seguimiento clínico continuo y alertas en tiempo real



Repositorio <https://github.com/cmorregof/homecare>
Bot https://t.me/project918_homecare_bot
Panel <https://homecare-bice-beta.vercel.app>

Índice

1. Qué es CARMEN	1
2. Arquitectura	2
2.1. Principios que no se negocian	2
2.2. Endpoints principales	2
3. Puesta en marcha	3
3.1. Variables de entorno	3
3.2. Backend	3
3.3. Frontend	3
3.4. Integración continua	3
4. Los tres espacios de rol	4
5. Cómo entra una medición	5
5.1. Por Telegram	5
5.2. Por el panel web	5
5.3. Qué hace el servidor con el dato	7
6. El sistema visual	8
6.1. Una desviación deliberada	8
6.2. Por qué los niveles no se distinguen solo por color	8
7. Modo demostración	9
8. Base de datos	10
8.1. Migración propuesta y no aplicada	10
9. Decisiones abiertas	11
A. Cómo compilar este documento	12

1. Qué es CARMEN

CARMEN es un sistema multiagente para el seguimiento domiciliario y el triaje de riesgo cardio-cerebrovascular en entornos de bajos recursos. Lleva el nombre de la enfermera de confianza del barrio: la persona a la que se le cuenta cómo se siente uno antes que a nadie.

El sistema hace tres cosas. Recibe signos vitales tomados en casa, estima un nivel de riesgo con un modelo entrenado y reglas clínicas duras, y avisa al equipo de la IPS cuando ese riesgo lo amerita. Alrededor de eso hay una conversación: CARMEN pregunta, entiende respuestas en lenguaje corriente y responde con la voz de una abuela que sabe de esto.

Tres puertas de entrada

Canal	Para quién	Estado
Bot de Telegram	Paciente en casa	En producción
Panel web	Paciente, IPS y administración	En producción
API REST (FastAPI)	Integraciones	En producción

Tabla 1: Formas de interactuar con el sistema.

No existe una aplicación móvil nativa: el canal móvil es el bot de Telegram, donde CARMEN ya conversa, guía la toma de signos y alerta al médico asignado.

2. Arquitectura

Pieza	Tecnología
Backend	FastAPI (Python 3.12)
Base de datos	Supabase (PostgreSQL con RLS)
Modelo de riesgo	LightGBM con explicabilidad SHAP
Pronóstico	CARMEN-Forecast (TinyTemporalTransformer)
Recuperación clínica	RAG sobre guías MINSALUD, NEWS2 y MEWS
Voz del agente	GPT-4o sobre un núcleo determinista
Frontend	Next.js 14 (App Router) y Tailwind CSS
Bot	python-telegram-bot
Despliegue	Railway (backend) y Vercel (frontend)

Tabla 2: Componentes del sistema.

2.1. Principios que no se negocian

Hay cuatro decisiones estructurales que el resto del sistema da por sentadas. Cambiarlas no es refactorizar, es construir otro producto.

1. **La portería de rol vive en el servidor.** El `middleware.ts` consulta `profiles.role` en Supabase antes de entregar cualquier ruta protegida. El cliente no decide qué puede ver.
2. **La sesión vive en una cookie `httpOnly`** leída en servidor. No hay tokens en `localStorage`.
3. **RLS es la frontera de seguridad**, no la interfaz. Una consulta mal hecha desde el cliente no devuelve datos ajenos porque la base lo impide.
4. **Los umbrales clínicos viven en un único sitio**, el backend (`risk_levels.py` y `nurse_agent.py`). Ninguna pantalla calcula un nivel de riesgo por su cuenta.

2.2. Endpoints principales

Ruta	Qué hace
POST <code>/agents/vital-report</code>	Recibe signos vitales, predice riesgo, guarda el reporte y dispara alerta si procede
POST <code>/agents/chat</code>	Conversación libre con CARMEN
POST <code>/ml/predict</code>	Predicción de riesgo sin efectos secundarios
POST <code>/ml/forecast</code>	Probabilidad de deterioro a 6, 12 y 24 horas
GET <code>/models</code>	Métricas de los modelos entrenados
POST <code>/telegram/webhook</code>	Entrada del bot
GET <code>/patients, /vitals, /alerts, /reports</code>	Lectura de entidades

Tabla 3: Superficie HTTP del backend.

3. Puesta en marcha

3.1. Variables de entorno

Se declaran en `.env.example`. El script `scripts/check_env.py` valida que estén completas antes de desplegar.

Variable	Para qué
SUPABASE_URL	Proyecto de Supabase
SUPABASE_SERVICE_KEY	Clave de servicio, solo backend
SUPABASE_ANON_KEY	Clave pública
OPENAI_API_KEY	Voz de CARMEN y reporte del médico
TELEGRAM_BOT_TOKEN	Bot
ML_MODEL_PATH	Bundle de LightGBM
NEXT_PUBLIC_SUPABASE_URL	Frontend: sesión y tiempo real
NEXT_PUBLIC_SUPABASE_ANON_KEY	Frontend: idem
NEXT_PUBLIC_API_URL	Frontend: dónde vive el backend

Tabla 4: Variables imprescindibles.

Cuidado con las dos últimas del frontend. Si `NEXT_PUBLIC_SUPABASE_URL` o su clave faltan o están mal, el panel *no falla*: sirve datos de demostración. Eso se explica en la sección 7.

3.2. Backend

```
# Con Docker (levanta también un PostgreSQL con pgvector)
docker compose up --build

# O directamente
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

3.3. Frontend

```
cd frontend
npm ci
npm run dev      # desarrollo, http://localhost:3000
npm run build    # produccion
npm run typecheck # tsc --noEmit
npm run lint     # next lint
```

3.4. Integración continua

Dos flujos en `.github/workflows/`. **Frontend CI/CD** corre `lint`, `build` y `typecheck`, y despliega a Vercel al integrar en `main`. **Backend CI/CD** compila, corre la suite de `pytest`, valida el entorno y construye la imagen Docker, con despliegue a Railway.

Conviene saber que **Tailwind descarta en silencio las clases que no reconoce**. Una clase mal escrita no rompe el `build` ni el `typecheck`: simplemente no pinta nada. Verde en CI no equivale a correcto en pantalla.

4. Los tres espacios de rol

El `middleware.ts` protege tres prefijos: `/patient`, `/ips` y `/admin`. Cualquier ruta fuera de ellos queda sin portería, así que toda pantalla que lea o escriba datos clínicos debe vivir bajo uno de los tres.

Rol	Ruta	Para qué
Paciente	<code>/patient/dashboard</code>	Riesgo actual, signos y tendencia
	<code>/patient/vitals/new</code>	Registrar una medición
	<code>/patient/history</code>	Historial paginado
	<code>/patient/chat</code>	Conversación con CARMEN
IPS	<code>/ips/dashboard</code>	Pacientes priorizados por riesgo
	<code>/ips/patients</code>	Listado asignado
	<code>/ips/patients/[id]</code>	Ficha clínica
	<code>/ips/alerts</code>	Centro de alertas en tiempo real
	<code>/ips/chat</code>	Consulta clínica con CARMEN
Administración	<code>/admin/dashboard</code>	Métricas del sistema
	<code>/admin/users</code>	Gestión de usuarios
	<code>/admin/models</code>	Rendimiento de los modelos
	<code>/admin/rag</code>	Documentos indexados
Público	<code>/login</code> , <code>/register</code>	Acceso

Tabla 5: Las quince rutas del panel.

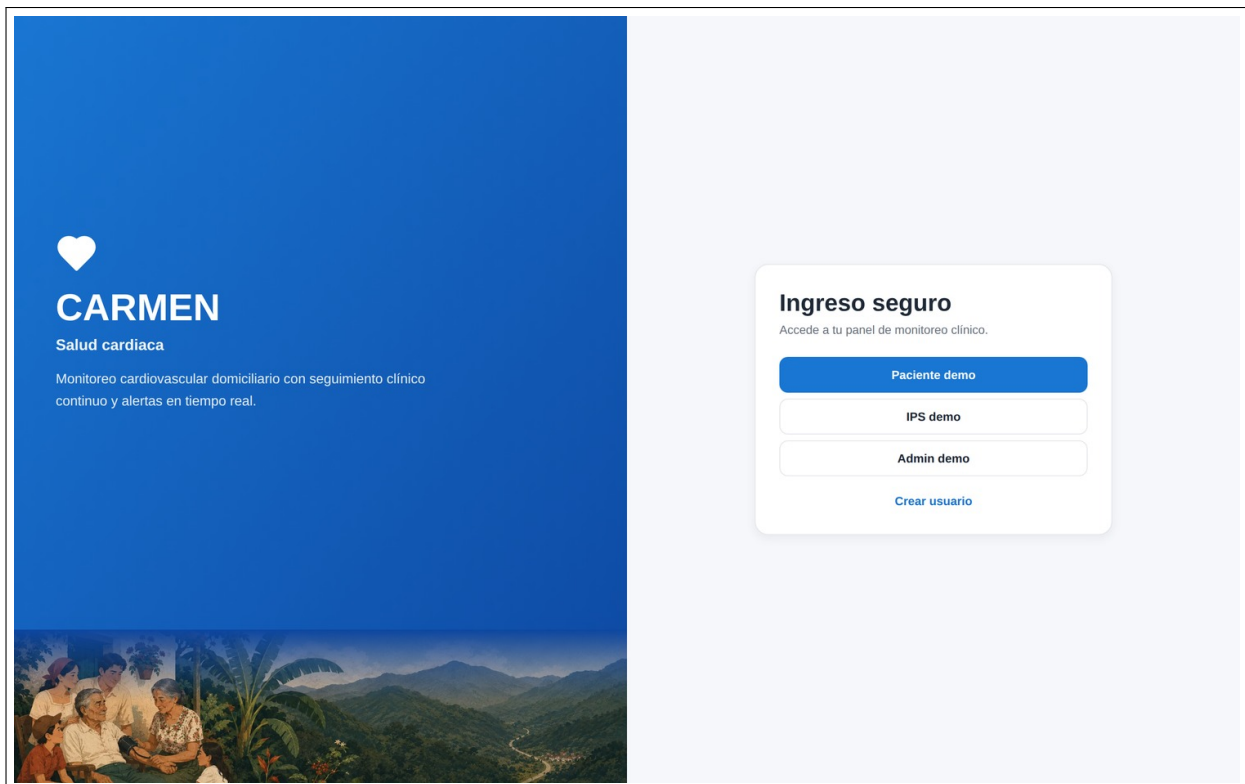


Figura 1: Pantalla de entrada. La ilustración de portada acompaña a la marca; el formulario cede su sitio a tres accesos de prueba cuando Supabase no está configurado.

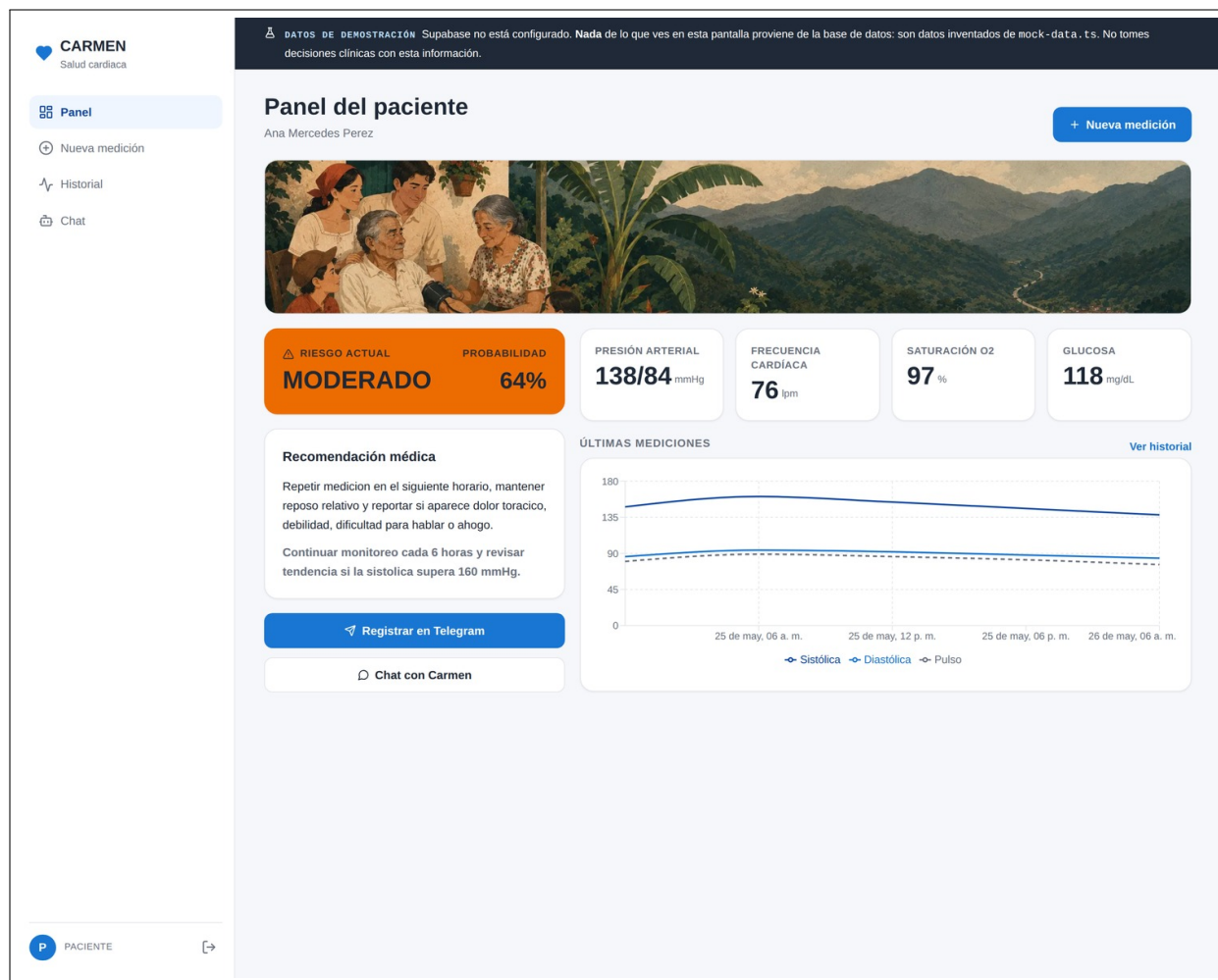


Figura 2: Panel del paciente. El nivel de riesgo y su probabilidad los decide el servidor; la pantalla solo los pinta.

5. Cómo entra una medición

5.1. Por Telegram

El paciente escribe al bot. CARMEN guía la toma paso a paso, acepta respuestas en lenguaje corriente (*noventa y siete, sí, así es*) y valida cada dato antes de seguir. Es el camino que ya usaban los pacientes antes de existir el formulario web.

5.2. Por el panel web

El formulario envía a `POST /agents/vital-report` con `source: "web"`. Cuatro decisiones merecen explicación.

Los rangos son los del servidor, no los del formulario. Están espejados del validador del servidor (`nurse_agent.py`) y el código lleva una nota que prohíbe estrecharlos localmente. La razón es concreta: un formulario más estrecho que la fisiología que vigila rechaza justo las mediciones que importan.

Se envía el instante completo, no la fecha. La columna `recorded_at` es `TIMESTAMP` y la cadencia de monitoreo es de seis horas: guardar solo el día perdería el orden de las tomas.

Incluye frecuencia respiratoria y temperatura, dos de los seis signos que el modelo de pronóstico usa.

No hay lógica clínica en el cliente. El formulario no calcula niveles ni colorea por umbral

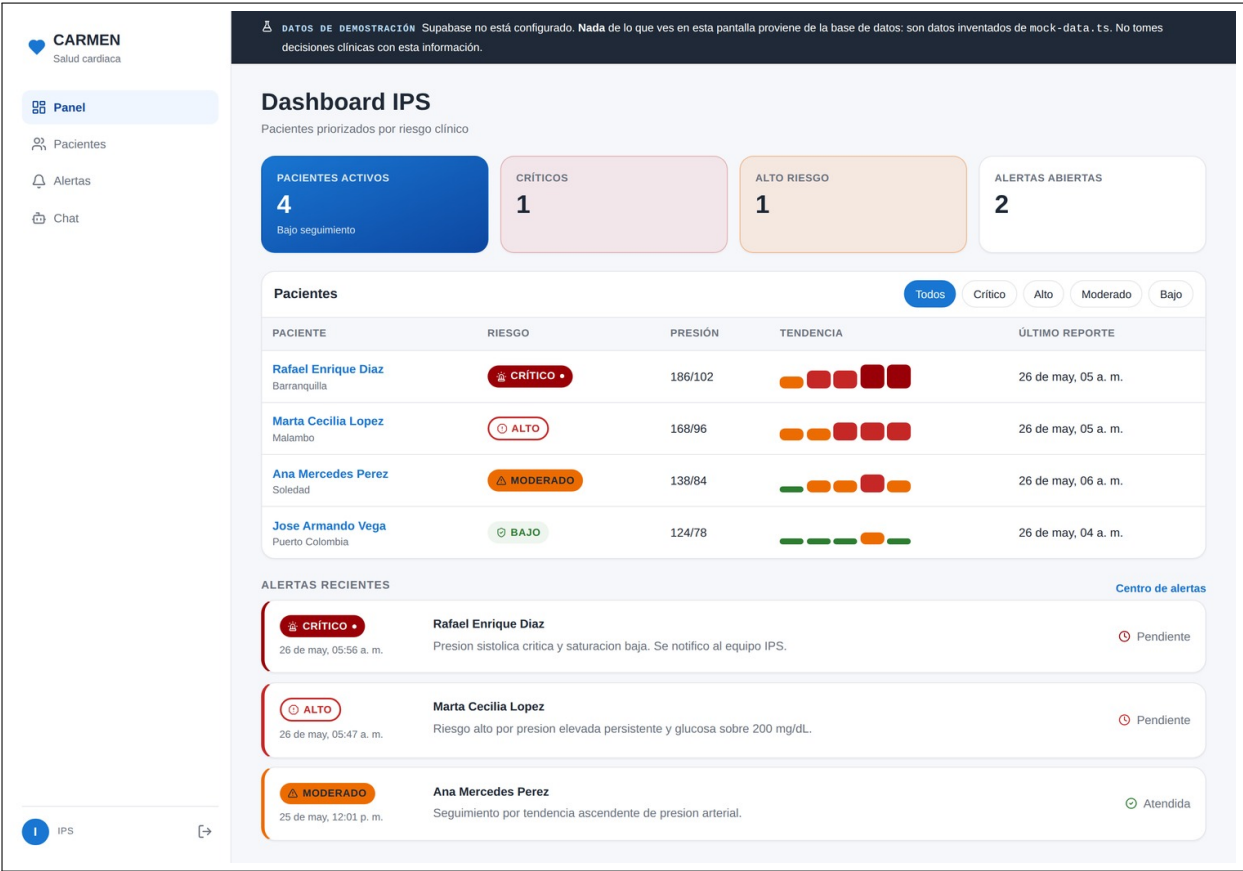


Figura 3: Panel de la IPS. Los pacientes se ordenan por gravedad y la columna de tendencia codifica el riesgo en la altura de la barra, no solo en el color.

Signo	Mínimo	Máximo
Presión sistólica (mmHg)	50	260
Presión diastólica (mmHg)	30	160
Frecuencia cardíaca (lpm)	25	220
Frecuencia respiratoria (rpm)	6	50
Saturación de oxígeno (%)	1	100
Temperatura (°C)	30	43
Glucosa (mg/dL)	20	600
Peso (kg)	25	300
Dolor, mareo y disnea	0	10

Tabla 6: Rangos de plausibilidad. Presión sistólica, diastólica y frecuencia cardíaca son obligatorias; el resto es opcional.

CARMEN

Salud cardíaca

Panel

Nueva medición

Historial

Chat

DATOS DE DEMOSTRACIÓN Supabase no está configurado. Nada de lo que ves en esta pantalla proviene de la base de datos: son datos inventados de mock-data.ts. No tomes decisiones clínicas con esta información.

Nueva medición

Registra tus signos vitales; Carmen evaluará el riesgo.

Momento de la medición

Fecha y hora

09/02/2026, 10:56 AM

Se guarda el instante completo, no solo el día.

Signos vitales

Presión sistólica (mmHg)

Presión diastólica (mmHg)

Frecuencia cardíaca (lpm)

Frecuencia respiratoria (rpm)

Saturación de oxígeno (%)

Temperatura (°C)

Glucosa (mg/dL)

Peso (kg)

Síntomas

De 0 (ninguno) a 10 (el peor que hayas sentido).

Dolor

Mareo

Dificultad para respirar

Notas (opcional)

Cualquier cosa que quieras que el equipo clínico sepa.

Figura 4: Formulario de captura en /patient/vitals/new.

propio. El nivel, la probabilidad y el texto que aparecen tras enviar son literalmente lo que devolvió el servidor.

5.3. Qué hace el servidor con el dato

1. Valida rangos y campos obligatorios.
2. Predice el riesgo con LightGBM, o con reglas clínicas si el modelo no carga.
3. Aplica **anulaciones duras**: sistólica ≥ 180 o < 80 , frecuencia cardíaca > 130 o < 40 , saturación < 88 , glucosa > 400 o < 50 . Cualquiera de ellas eleva el caso a crítico sin importar qué dijo el modelo.
4. Redacta el reporte clínico apoyado en las guías indexadas.
5. Notifica al médico asignado por Telegram si el riesgo lo amerita.

6. El sistema visual

La identidad visual está portada del trabajo del equipo de `salud-cardiaca-web`: colores, radios, sombras y escala tipográfica salen de su hoja de estilos. Se tomaron valores y estructura de marcado; ninguna línea de su código entró al repositorio.

6.1. Una desviación deliberada

En el original, el color primario y el de peligro son prácticamente el mismo rojo. Allí no importa porque su producto no tiene niveles de riesgo. Aquí el rojo *significa* crítico, así que el primario se desplazó al azul y el rojo quedó reservado por completo para gravedad.

Token	Valor	Uso
<code>brand</code>	 #1976D2	Botón primario, enlaces, activo
<code>brand.dark</code>	 #0D47A1	Degradados y estados
<code>risk.low</code>	 #2E7D32	Riesgo bajo
<code>risk.moderate</code>	 #ED6C02	Riesgo moderado
<code>risk.high</code>	 #C62828	Riesgo alto
<code>risk.critical</code>	 #9A0007	Riesgo crítico

Tabla 7: Colores con significado. Radios 6/10/16/24 px, tres sombras y base tipográfica de 14 px completan el sistema.

6.2. Por qué los niveles no se distinguen solo por color

`risk.high` y `risk.critical` son dos rojos oscuros. El contraste entre ellos es de **1,57:1**, y con deuteranopía es aún menor. Quien no separe esos dos tonos no podría distinguir un paciente de riesgo alto de uno crítico: la distinción más consecuente del producto.

Por eso cada nivel difiere en cuatro canales a la vez.

Nivel	Relleno	Texto	Peso	Contraste
Bajo	tinte al 8 %	verde	600	4,63:1
Moderado	sólido	oscuro	600	4,72:1
Alto	contorno de 2 px	rojo	700	5,62:1
Crítico	sólido	blanco	800	8,83:1

Tabla 8: Los cuatro niveles superan WCAG AA, medido sobre el DOM renderizado. Cada uno lleva además un icono distinto.

Leídos en escala de grises, los rellenos van de claro a muy oscuro y la polaridad texto/relleno se invierte dos veces, de modo que el orden sobrevive sin ninguna información de tono. El par alto/crítico, que el color no puede separar, queda a **8,83:1** como tratamientos.

PACIENTE	RIESGO	PRESIÓN	TENDENCIA	ÚLTIMO REPORTE
Rafael Enrique Diaz Barranquilla	 CRÍTICO	186/102		26 de may, 05 a. m.
Marta Cecilia Lopez Malambo	 ALTO	168/96		26 de may, 05 a. m.
Ana Mercedes Perez Soledad	 MODERADO	138/84		26 de may, 06 a. m.
Jose Armando Vega Puerto Colombia	 BAJO	124/78		26 de may, 04 a. m.

Figura 5: Los cuatro niveles en visión normal.

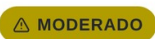
PACIENTE	RIESGO	PRESIÓN	TENDENCIA	ÚLTIMO REPORTE
Rafael Enrique Diaz Barranquilla	 CRÍTICO	186/102		26 de may, 05 a. m.
Marta Cecilia Lopez Malambo	 ALTO	168/96		26 de may, 05 a. m.
Ana Mercedes Perez Soledad	 MODERADO	138/84		26 de may, 06 a. m.
Jose Armando Vega Puerto Colombia	 BAJO	124/78		26 de may, 04 a. m.

Figura 6: Los mismos, simulando deuteranopía. Los distintivos siguen siendo distinguibles; la columna de tendencia también, porque codifica la gravedad en la altura de la barra.

7. Modo demostración

El panel puede servir datos inventados en dos situaciones distintas, y solo una tiene que ver con la configuración.

Primera: Supabase no está configurado. Todas las lecturas retroceden a `lib/mock-data.ts`. Una variable de entorno mal puesta en producción deja el producto entero en este estado.

Segunda, y más peligrosa: Supabase está configurado y sano, y aun así hay datos inventados. El listado de pacientes de la IPS, las alertas de cada ficha y todo el panel de administración salen de los mocks sin importar la configuración.

Un aviso que solo mirara la configuración habría permanecido oculto justamente en el segundo caso. Por eso el banner nombra *qué* conjuntos son ficticios, y se acompaña de un `console.warn` en cada una de las ocho ramas que sirven mocks.

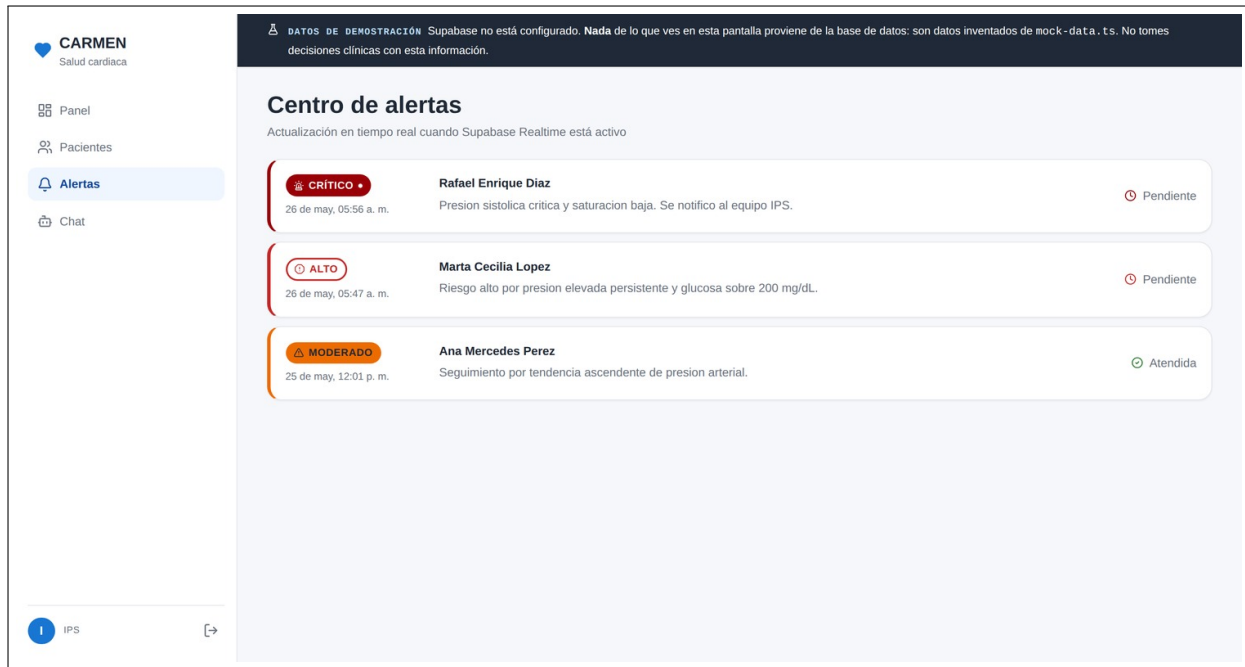


Figura 7: El banner de datos de demostración. Evita deliberadamente los tonos de peligro y advertencia: esos significan «este paciente está en riesgo», y este significa «nada de esto es un paciente».

8. Base de datos

Tabla	Contenido
profiles	Persona y rol; enlaza con <code>auth.users</code>
ips	Instituciones prestadoras
patient_clinical_info	Antecedentes y línea base
vital_signs	Cada toma, con su origen y su instante
risk_predictions	Nivel, probabilidad y factores
clinical_reports	Interpretación y recomendaciones
alerts	Avisos al equipo clínico
rag_documents	Guías clínicas indexadas

Tabla 9: Esquema principal, en `backend/db/schemas.sql`.

8.1. Migración propuesta y no aplicada

`backend/db/migrations/20260902_add_demographics.sql` añade a `profiles` siete columnas de contexto colombiano: EPS, grupo étnico según categorías DANE, clasificación de insuficiencia cardiaca A–D, fecha de nacimiento, sexo biológico, departamento y ciudad. Es idempotente y está verificada aplicándola dos veces contra un PostgreSQL desechable.

Antes de aplicarla hay que resolver un solapamiento. `patient_clinical_info` ya guarda `age` y `gender`, que se solapan con `birth_date` y `biological_sex`. Aplicarla tal cual deja dos fuentes para el mismo hecho, libres de divergir. Y una edad almacenada es correcta el día que se escribe y falsa a partir del siguiente cumpleaños. La migración solo añade columnas: resolver el solapamiento es una decisión aparte.

9. Decisiones abiertas

1. **El solapamiento demográfico.** Poblar `birth_date`, deprecar `age` y calcular la edad al consultar, o mantener las dos fuentes conscientemente.
2. **Aplicación móvil nativa.** Hoy el canal móvil es Telegram. Una app nativa no lo extiende, lo duplica; habría que decidir si conviven.
3. **Los datos que el panel muestra y la base no tiene.** El nombre del usuario en la barra lateral y los umbrales de la gráfica de tendencia están preparados pero sin cablear, a la espera de que alguien los sirva.
4. **El buscador de pacientes** de la IPS es decorativo: no está conectado a ninguna consulta.

A. Cómo compilar este documento

```
cd docs/manual
pdflatex manual.tex
pdflatex manual.tex # la segunda pasada resuelve el indice
```

Requiere una instalación estándar de TeX Live. En Debian o Ubuntu:

```
sudo apt-get install texlive-latex-base texlive-latex-recommended \
texlive-fonts-recommended texlive-lang-spanish lmodern
```

El documento usa solo `graphicx`, `xcolor`, `geometry`, `hyperref`, `booktabs`, `fancyhdr`, `listings`, `babel` y `lmodern`, todos incluidos en esos paquetes. No requiere `titlesec`, `tcolorbox` ni `enumitem`. Las figuras están en `figuras/` y son capturas de la aplicación en ejecución.